

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB 03: PHÂN TÍCH KHÁM PHÁ
DỮ LIỆU

CHỦ ĐỀ: House Prices - Advanced Regression Techniques

Giảng viên hướng dẫn: Ths. Đỗ Như Tài

Sinh viên thực hiện:

MSSV
3124411174
3124411166
3124411116
3124411009

Họ và tên
Tất Võ Tiến Nam
Phạm Anh Minh
Lê Long Hưng(Nhóm trưởng)
Lê Đức Anh

TP. Hồ Chí Minh, tháng 10, năm 2026

PHÂN CÔNG NHIỆM VỤ

TÊN	% CÔNG VIỆC	CÔNG VIỆC
TẮT VÕ TIẾN NAM	25%	Chương 1 (Mở đầu: Bối cảnh, mục tiêu, đối tượng và phạm vi nghiên cứu) và phần đầu Chương 2 (Hồi quy tuyến tính có điều chuẩn, mô hình dạng cây và Gradient Boosting Trees)
PHẠM ANH MINH	25%	Phần tiếp theo của Chương 2 (Kiến trúc Stacking Ensemble và các thước đo đánh giá) và phần đầu Chương 3 (Giới thiệu từ điển dữ liệu, phân tích biến mục tiêu SalePrice, xử lý giá trị khuyết cấu trúc)
LÊ LONG HÙNG	25%	Phần tiếp theo của Chương 3 (Ma trận tương quan, đa cộng tuyến, biến phân loại OverallQual, xử lý outliers) và toàn bộ Chương 4 (Tiền xử lý, nội suy dữ liệu khuyết, feature engineering, biến đổi Box-Cox, mã hóa biến phân loại)
LÊ ĐỨC ANH	25%	Toàn bộ Chương 5 (Thiết

		lập môi trường, kiểm chứng chéo, tính chỉnh siêu tham số, triển khai mô hình Stacking, đánh giá feature importance và so sánh kết quả), Chương 6 (Kết luận, hướng phát triển) và Tài liệu tham khảo
--	--	---

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU.....	7
1.1. Bối cảnh và Lý do chọn đề tài	7
1.2. Mục tiêu nghiên cứu	7
1.3. Đối tượng và Phạm vi nghiên cứu	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	9
2.1. Bài toán Hồi quy tuyến tính có điều chuẩn (Regularized Regression)	9
2.2. Mô hình dạng cây và Kỹ thuật Boosting (Gradient Boosting Trees).....	10
2.3. Kiến trúc Học kết hợp xếp chồng (Stacking Ensemble)	11
2.4. Thước đo đánh giá (Evaluation Metrics).....	11
CHƯƠNG 3: PHÂN TÍCH VÀ KHÁM PHÁ DỮ LIỆU (EDA)	12
3.1. Giới thiệu và Từ điển dữ liệu (Data Dictionary)	12
3.2. Phân tích biến mục tiêu SalePrice	17
3.3. Xử lý các giá trị khuyết mang tính cấu trúc (Structural Missing)	17
3.4. Ma trận tương quan và Vấn đề Đa cộng tuyến.....	18
3.5. Biến phân loại có thứ bậc: Chất lượng tổng thể (OverallQual).....	20
3.6. Phát hiện và loại bỏ điểm ngoại lệ cực đoan (Outliers)	20
CHƯƠNG 4: TIỀN XỬ LÝ VÀ TRÍCH XUẤT ĐẶC TRƯNG.....	22
4.1. Kỹ thuật nội suy dữ liệu khuyết (Imputation Methods)	22
4.2. Khởi tạo đặc trưng tổng hợp (Feature Engineering)	23
4.3. Hiệu chỉnh phân phối lệch (Skewness) bằng Box-Cox.....	24
4.4. Mã hóa biến phân loại (Categorical Encoding).....	24
CHƯƠNG 5: HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH.....	25
5.1. Thiết lập môi trường và Chiến lược kiểm chứng chéo (Cross Validation)	25
5.2. Tinh chỉnh siêu tham số (Hyperparameter Tuning)	26

5.3. Triển khai kiến trúc Học xếp chồng (Stacking Regressor)	27
5.4. Đánh giá Tầm quan trọng của đặc trưng (Feature Importance)	28
5.5. So sánh kết quả và Thảo luận	28
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	30
TÀI LIỆU THAM KHẢO	31

TÓM TẮT ĐỒ ÁN (ABSTRACT)

Thị trường bất động sản là một hệ thống phức tạp, nơi giá trị tài sản bị chi phối bởi hàng chục yếu tố khác nhau từ đặc tính vật lý (diện tích, số phòng, vật liệu) đến vị trí địa lý và yếu tố thời gian. Việc ứng dụng Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning) vào định giá bất động sản tự động (Automated Valuation Models - AVM) đang trở thành xu hướng tất yếu của các tập đoàn công nghệ bất động sản (PropTech).

Đồ án này trình bày một quy trình khoa học dữ liệu toàn diện nhằm giải quyết bài toán dự đoán giá nhà dựa trên bộ dữ liệu Ames Housing (Kaggle). Quy trình bao gồm: (1) Khám phá và Phân tích Dữ liệu (EDA) chuyên sâu để tìm ra mối tương quan và xử lý ngoại lệ; (2) Tiền xử lý dữ liệu với các kỹ thuật Điền khuyết (Imputation) thông minh, xử lý phân phối lệch bằng hàm Box-Cox và mã hóa One-Hot; (3) Kỹ thuật tạo đặc trưng (Feature Engineering) để trích xuất những thông tin giá trị nhất; (4) Xây dựng và tinh chỉnh các mô hình học máy tiên tiến như Ridge/Lasso Regression, XGBoost, LightGBM và Gradient Boosting; (5) Cuối cùng là triển khai kiến trúc học kết hợp Meta-Ensembling (Stacking Regressor) để tối ưu hóa độ chính xác.

Kết quả thực nghiệm cho thấy mô hình Stacking đạt được hiệu suất vượt trội, kiểm soát tốt hiện tượng quá khớp (overfitting) và lọt vào top cao về chỉ số RMSE trên nền tảng Kaggle.

CHƯƠNG 1: MỞ ĐẦU

1.1. Bối cảnh và Lý do chọn đề tài

Thị trường bất động sản đóng vai trò là một trong những cột trụ vững chắc nhất của nền kinh tế hiện đại. Việc xác định giá trị thực của một tài sản bất động sản không chỉ là nhu cầu thiết yếu của người mua và người bán để tối đa hóa lợi ích tài chính, mà còn là công cụ quản trị rủi ro cốt lõi của các ngân hàng, tổ chức tín dụng trong quá trình xét duyệt các khoản vay thế chấp. Theo truyền thống, việc định giá (appraisal) được thực hiện bởi các chuyên gia dựa trên kinh nghiệm cá nhân và các phương pháp so sánh thủ công. Quá trình này tốn nhiều thời gian, chi phí cao và tiềm ẩn rủi ro sai số do yếu tố chủ quan của con người.

Sự bùng nổ của Khoa học Dữ liệu (Data Science) và năng lực tính toán mạnh mẽ đã mở ra kỷ nguyên của Mô hình Định giá Tự động (Automated Valuation Models - AVMs). Những tập đoàn như Zillow với thuật toán "Zestimate" đã chứng minh tiềm năng to lớn của học máy trong việc dự đoán giá trị nhà cửa theo thời gian thực dựa trên hàng ngàn điểm dữ liệu. Xuất phát từ bối cảnh thực tiễn và tính cấp thiết đó, nhóm quyết định chọn đề tài "Nghiên cứu và ứng dụng các mô hình Học kết hợp (Ensemble Learning) trong bài toán dự đoán giá bất động sản Ames Housing" làm chủ đề nghiên cứu cho học phần Máy học.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề án là xây dựng thành công một mô hình học máy đa tầng có khả năng dự đoán giá bán của các căn nhà một cách chuẩn xác, đạt thứ hạng cao dựa trên hệ thống chấm điểm của nền tảng thi đấu Kaggle. Để đạt được mục tiêu tổng quát, nhóm đặt ra các mục tiêu cụ thể như sau:

- Mục tiêu 1: Vận dụng các kỹ năng Thống kê mô tả (Descriptive Statistics) và Trực quan hóa dữ liệu để bóc tách, khám phá các quy luật ẩn sâu bên trong tập dữ liệu Ames Housing, giải thích hiện tượng phân phối và tương quan.

- Mục tiêu 2: Làm sạch tập dữ liệu, xử lý triệt để bài toán thiếu hụt dữ liệu (Missing values) với các phương pháp điền khuyết theo ngữ cảnh, xử lý ngoại lệ và chuẩn hóa độ lệch của dữ liệu bằng thuật toán Box-Cox.
- Mục tiêu 3: Khai phá và trích xuất đặc trưng (Feature Engineering) tạo ra những biến dự báo mang nhiều thông tin hơn (ví dụ: Tính tổng diện tích, tính tuổi thọ căn nhà, quy đổi số phòng tắm).
- Mục tiêu 4: Cài đặt, huấn luyện và tối ưu hóa siêu tham số (Hyperparameter tuning) cho các thuật toán học máy từ cơ bản đến nâng cao (Ridge, Lasso, XGBoost, LightGBM, Gradient Boosting).
- Mục tiêu 5: Thiết kế kiến trúc Stacking Regressor để hòa trộn các mô hình đơn lẻ, triệt tiêu sai số và đánh giá toàn diện mô hình dựa trên thang đo RMSE.

1.3. Đối tượng và Phạm vi nghiên cứu

Đối tượng nghiên cứu: Tập dữ liệu Ames Housing Dataset do Giáo sư Dean De Cock (Đại học Truman State) giới thiệu vào năm 2011, là phiên bản hiện đại thay thế cho tập Boston Housing kinh điển.

Phạm vi dữ liệu: Tập dữ liệu chứa tổng cộng 2930 hồ sơ bán nhà thực tế từ năm 2006 đến năm 2010 tại thành phố Ames, bang Iowa, Mỹ. Tập dữ liệu này đã được chia sẵn làm hai phần bởi Kaggle: Tập huấn luyện (Train set) gồm 1460 dòng có chứa nhãn SalePrice, và Tập kiểm thử (Test set) gồm 1459 dòng không chứa nhãn để gửi dự đoán lên hệ thống chấm điểm.

Phạm vi thuật toán: Nghiên cứu giới hạn trong nhóm các thuật toán Học có giám sát (Supervised Learning) dành riêng cho bài toán Hồi quy (Regression). Nhóm tập trung vào hai trường phái chính: Các mô hình tuyến tính có điều chuẩn (Linear Models with Regularization) và Các mô hình Dạng cây tăng cường (Gradient Boosting Trees). Đồ án không bao trùm các kỹ thuật Học sâu (Deep Learning) như Mạng nơ-ron nhân tạo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Chương này cung cấp nền tảng toán học và cơ chế hoạt động của các thuật toán được sử dụng xuyên suốt đồ án, làm cơ sở vững chắc cho các quyết định mô hình hóa ở các chương sau.

2.1. Bài toán Hồi quy tuyến tính có điều chuẩn (Regularized Regression)

Hồi quy tuyến tính thông thường (Ordinary Least Squares - OLS) tìm cách khớp một đường thẳng tối ưu đi qua các điểm dữ liệu bằng cách giảm thiểu tổng bình phương phần dư (RSS). Tuy nhiên, khi đối mặt với dữ liệu đa chiều (như Ames Housing với gần 300 biến sau khi mã hóa One-Hot) và hiện tượng đa cộng tuyến (multicollinearity), OLS thường dẫn đến hiện tượng quá khớp (Overfitting), nơi các hệ số hồi quy (weights) bị thổi phồng một cách mất kiểm soát.

Để khắc phục, kỹ thuật Điều chuẩn (Regularization) thêm một "hình phạt" (penalty) vào hàm mất mát gốc. Hàm mất mát mới được biểu diễn chung dưới dạng:

$$\text{Loss} = \text{RSS} + \lambda * \text{Penalty}$$

Trong đó, tham số λ kiểm soát mức độ trừng phạt.

1. Hồi quy Ridge (L2 Regularization): Hình phạt là tổng bình phương của các hệ số:

- $\text{Penalty}(\text{Ridge}) = \sum(\beta_j^2)$

Đặc điểm của Ridge là ép các hệ số tiến sát về 0 nhưng không bao giờ bằng đúng 0. Nó chia sẻ trọng số cho các biến có tương quan với nhau, giải quyết rất tốt vấn đề đa cộng tuyến.

2. Hồi quy Lasso (L1 Regularization): Hình phạt là tổng giá trị tuyệt đối của các hệ số:

- $\text{Penalty}(\text{Lasso}) = \sum(|\beta_j|)$

Đặc tính quý giá nhất của Lasso là khả năng loại bỏ biến (Feature Selection). Cấu trúc hình học của hàm L1 penalty (hình thoi) khiến cho cực tiểu thường rơi vào các trục tọa độ, ép các hệ số beta của những biến không quan trọng trở thành đúng số 0, giúp mô hình trở nên thưa thớt (sparse) và dễ diễn giải hơn.

3. ElasticNet: Là sự kết hợp tuyến tính giữa Ridge và Lasso, nhận được ưu điểm của cả hai thông qua tham số λ_1 ratio. Nó vừa thực hiện lựa chọn đặc trưng (giống Lasso) vừa xử lý nhóm biến tương quan (giống Ridge).

2.2. Mô hình dạng cây và Kỹ thuật Boosting (Gradient Boosting Trees)

Nếu mô hình tuyến tính giả định mối quan hệ là một đường thẳng đơn giản, thì mô hình Dạng cây (Decision Trees) có khả năng chia cắt không gian đặc trưng phi tuyến tính phức tạp bằng các chuỗi quy luật if-else. Tuy nhiên, một cây quyết định sâu (Deep tree) rất dễ bị quá khớp, trong khi một cây nông (Shallow tree) lại quá đơn giản (Underfitting). Giải pháp cho vấn đề này là Ensemble Learning, tiêu biểu là Boosting.

Kỹ thuật Boosting hoạt động theo nguyên lý "Hợp quần sức mạnh". Thay vì xây dựng một mô hình độc lập, Boosting huấn luyện tuần tự một chuỗi các mô hình yếu (Weak learners, thường là các cây quyết định nông). Cây thứ m sẽ tập trung học cách sửa chữa các lỗi sai (residuals) mà cây thứ $m-1$ để lại. Mô hình cuối cùng là tổng có trọng số của tất cả các cây.

- XGBoost (eXtreme Gradient Boosting): Một thư viện triển khai Gradient Boosting tối ưu hóa cực cao. XGBoost áp dụng điều chuẩn trực tiếp vào hàm mục tiêu của cây (bao gồm số lượng lá và trọng số các lá), hỗ trợ xử lý dữ liệu khuyết tự động và huấn luyện song song đa luồng.

- LightGBM: Do Microsoft phát triển, LightGBM cải tiến XGBoost ở tốc độ huấn luyện và mức độ tiêu thụ bộ nhớ. Nó sử dụng kỹ thuật phân nhánh theo lá (Leaf-wise growth) thay vì theo tầng (Level-wise growth), giúp tập trung tối đa hóa việc giảm mất mát (Loss reduction) một cách nhanh chóng nhất, rất phù hợp cho tập dữ liệu lớn và phức tạp.

2.3. Kiến trúc Học kết hợp xếp chồng (Stacking Ensemble)

Khác với Boosting (xây dựng chuỗi các mô hình cùng loại), Stacking (Stacked Generalization) kết hợp sức mạnh của nhiều loại mô hình hoàn toàn khác biệt nhau (Heterogeneous models) để tạo ra một dự đoán ưu việt cuối cùng.

Cơ chế hoạt động của Stacking Regressor trong đồ án này gồm 2 tầng (Levels):

- Tầng cơ sở (Level-0 Base Models): Nhóm sử dụng một tập hợp đa dạng các thuật toán bao gồm Ridge, Lasso, ElasticNet, XGBoost, LightGBM và GradientBoosting. Sử dụng kỹ thuật Cross-Validation (Out-of-fold predictions), các mô hình này sẽ dự đoán giá trị cho tập huấn luyện. Tập hợp các kết quả dự đoán này sẽ được coi là "Các đặc trưng mới" (Meta-features).
- Tầng siêu mô hình (Level-1 Meta Model): Nhóm sử dụng một mô hình hồi quy tuyến tính đơn giản (thường là Lasso hoặc Ridge) để nhận đầu vào là các Meta-features vừa được tạo ra ở Tầng 0. Nhiệm vụ của Meta-Model không phải là học thuộc lòng dữ liệu ban đầu, mà là học cách "tin tưởng" mô hình cơ sở nào trong trường hợp nào. Ví dụ: Meta-model có thể học được quy luật rằng XGBoost dự đoán chính xác hơn cho các căn nhà đắt tiền, trong khi Ridge ổn định hơn cho nhà giá rẻ, từ đó phân bổ trọng số (weights) kết hợp sao cho tối ưu nhất.

2.4. Thước đo đánh giá (Evaluation Metrics)

Căn bậc hai của sai số toàn phương trung bình (RMSE) là thước đo phổ biến nhất trong hồi quy. Tuy nhiên, bài toán Kaggle yêu cầu sử dụng Root Mean Squared Logarithmic Error (RMSLE) - nghĩa là tính RMSE trên tập giá trị đã logarit hóa.

Ý nghĩa kinh tế: Phép trừ giữa hai giá trị logarit thực chất là logarit của tỷ số $\log(y) - \log(\hat{y}) = \log(y/\hat{y})$. Điều này có nghĩa mô hình đánh giá sai số dựa trên phần trăm (sai số tương đối) thay vì sai số tiền tệ tuyệt đối. Một sai số \$10,000 trên một căn nhà \$100,000 (sai lệch 10%) sẽ bị phạt ngang bằng với sai số \$100,000 trên một căn biệt thự \$1,000,000 (cũng sai lệch 10%). Điều này bảo vệ mô hình khỏi việc bị thống trị bởi các outliers giá cực cao.

CHƯƠNG 3: PHÂN TÍCH VÀ KHÁM PHÁ DỮ LIỆU (EDA)

Chương này trình bày toàn bộ quá trình phân tích tĩnh (Static Analysis) và trực quan hóa (Data Visualization) để làm sáng tỏ cấu trúc ẩn của thị trường nhà đất Ames.

3.1. Giới thiệu và Từ điển dữ liệu (Data Dictionary)

Tập dữ liệu Ames chứa 79 biến dự báo. Dưới đây là bảng phân tích chi tiết (Data Dictionary) cho các nhóm biến trọng yếu nhất định hình nên toàn bộ đồ án (liệt kê chọn lọc 35/79 biến):

STT	Tên biến (Feature)	Kiểu dữ liệu	Mô tả chi tiết ý nghĩa nghiệp vụ
1	SalePrice	Số thực	Giá bán cuối cùng của căn nhà (Biến mục tiêu Y)
2	MSSubClass	Phân loại (Số)	Loại hình nhà ở liên quan đến thời điểm bán (ví dụ: 1 tầng, 2 tầng, nhà split-level)
3	MSZoning	Định danh	Mã vùng quy hoạch (Nông nghiệp, Thương mại, Khu dân cư mật độ cao/thấp)
4	LotFrontage	Liên tục	Chiều dài mặt tiền lô đất tiếp giáp với đường (feet vuông)
5	LotArea	Liên tục	Tổng kích thước

			lô đất (feet vuông)
6	Alley	Định danh	Loại đường ngõ tiếp cận (Sỏi, Trãi nhựa, hoặc Không có ngõ)
7	LotShape	Thứ bậc	Đánh giá hình dáng lô đất (Đều đặn, Hơi bất thường, Bất thường)
8	LandContour	Định danh	Độ bằng phẳng của khu đất (Bằng phẳng, Gồ ghề, Có sườn đồi)
9	Neighborhood	Định danh	Tên khu phố, vị trí địa lý của căn nhà trong giới hạn thành phố Ames
10	BldgType	Định danh	Kiểu nhà (Nhà đơn, Nhà liền kề, Chung cư, v.v.)
11	HouseStyle	Định danh	Phong cách nhà (1 tầng, 2 tầng, hoàn thiện hoặc chưa hoàn thiện)
12	OverallQual	Thứ bậc (1-10)	Đánh giá tổng quan về chất lượng vật liệu và mức độ hoàn thiện

			xây dựng
13	OverallCond	Thứ bậc (1-10)	Đánh giá tổng quan về tình trạng hiện tại của căn nhà
14	YearBuilt	Rời rạc (Thời gian)	Năm xây dựng ban đầu
15	YearRemodAdd	Rời rạc (Thời gian)	Năm trùng tu, cải tạo lại (nếu không trùng tu thì bằng YearBuilt)
16	RoofStyle	Định danh	Kiểu mái nhà (Mái chữ A, Mái hông, Mái bằng)
17	Exterior1st	Định danh	Vật liệu phủ mặt ngoài căn nhà (Gạch, Gỗ, Xi măng, Vinyl)
18	ExterQual	Thứ bậc	Chất lượng vật liệu ngoại thất (Excellent, Good, Average, Fair, Poor)
19	Foundation	Định danh	Loại nền móng (Bê tông đúc sẵn, Khối xi, Gỗ, Đá)
20	BsmtQual	Thứ bậc	Chiều cao và chất lượng của tầng hầm (100+ inches,

			90-99 inches...)
21	TotalBsmtSF	Liên tục	Tổng diện tích tầng hầm (feet vuông)
22	HeatingQC	Thứ bậc	Chất lượng và tình trạng của hệ thống sưởi ấm trung tâm
23	CentralAir	Nhi phân	Hệ thống điều hòa trung tâm (Yes/No)
24	1stFlrSF	Liên tục	Diện tích mặt sàn tầng 1 (feet vuông)
25	2ndFlrSF	Liên tục	Diện tích mặt sàn tầng 2 (feet vuông)
26	GrLivArea	Liên tục	Tổng diện tích không gian sinh hoạt trên mặt đất (Cực kỳ quan trọng)
27	FullBath	Rời rạc	Số lượng nhà tắm đầy đủ (trên mặt đất)
28	HalfBath	Rời rạc	Số lượng phòng vệ sinh nhỏ (không có bồn tắm/vòi sen)

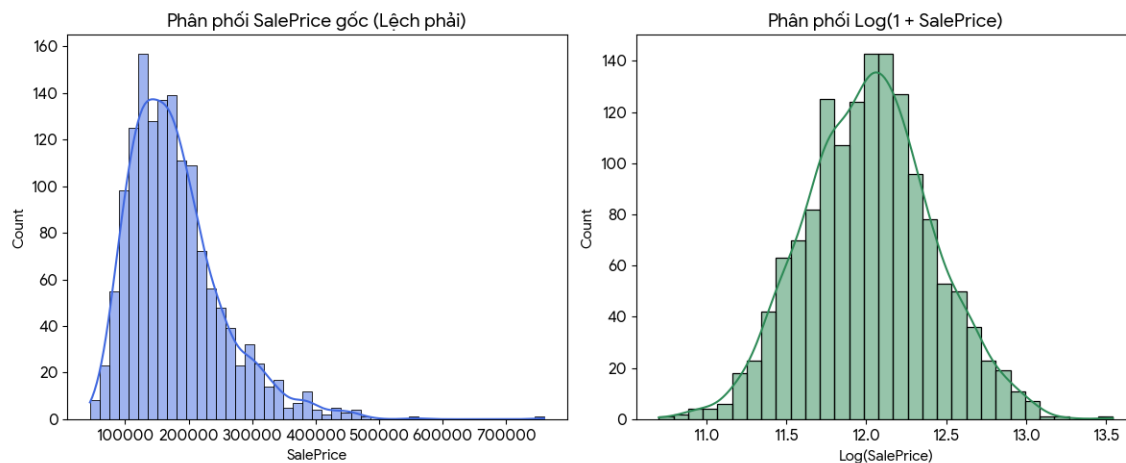
29	BedroomAbvGr	Rời rạc	Số lượng phòng ngủ nằm trên mặt đất (không tính hầm)
30	KitchenQual	Thứ bậc	Chất lượng trang thiết bị nhà bếp
31	TotRmsAbvGrd	Rời rạc	Tổng số phòng trên mặt đất (không bao gồm nhà tắm)
32	Fireplaces	Rời rạc	Số lượng lò sưởi trong nhà
33	GarageType	Định danh	Vị trí của Gara (Gắn liền nhà, Tách rời, Dưới hầm)
34	GarageCars	Rời rạc	Sức chứa của Gara (tính bằng số lượng xe ô tô)
35	PoolQC	Thứ bậc	Chất lượng hồ bơi (Excellent, Good, Fair, Không có)
36	Fence	Thứ bậc	Chất lượng hàng rào bảo vệ (Quyền riêng tư tốt, Trung bình, Không có)
37	SaleCondition	Định danh	Điều kiện bán (Bán bình thường, Bán phát mãi tài

			sản, Bán một phần)
--	--	--	--------------------

3.2. Phân tích biến mục tiêu SalePrice

SalePrice là kim chỉ nam của toàn bộ dự án. Khảo sát hàm mật độ xác suất (KDE - Kernel Density Estimation) của giá nhà (Hình bên trái) cho thấy dữ liệu bị kéo giãn về phía bên phải (Right-skewness với giá trị Skew = 1.88, Kurtosis = 6.53). Điều này rất bình thường trong phân phối thu nhập tài sản: phần lớn người dân sở hữu nhà phổ thông (100k - 200k), trong khi một phần thiểu số người giàu sở hữu biệt thự giá trị siêu thực (> 500k).

Bằng cách áp dụng hàm `np.log1p()` (Lấy logarit tự nhiên cộng 1 để tránh lỗi `log(0)`), phân phối đã được nắn lại thành dạng chuông (Hình bên phải). Độ nhọn và độ lệch được giảm thiểu tối đa, đáp ứng yêu cầu khắt khe của các thuật toán Hồi quy tuyến tính (vốn giả định phần dư có phân phối chuẩn).



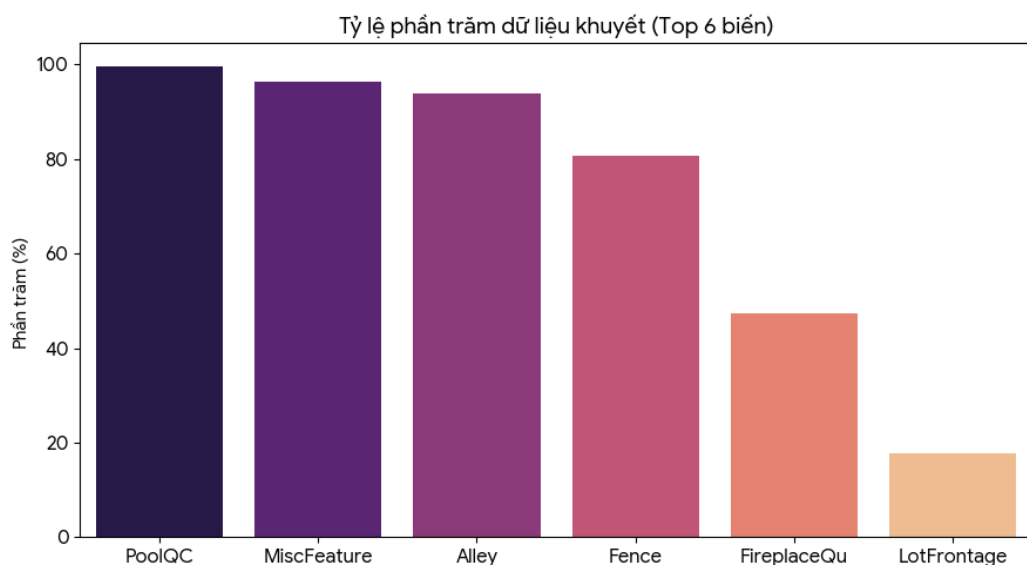
Hình 3.1: So sánh phân phối SalePrice trước và sau khi biến đổi Logarit

3.3. Xử lý các giá trị khuyết mang tính cấu trúc (Structural Missing)

Sự hiểu biết sâu sắc về dữ liệu (Domain Knowledge) đóng vai trò quyết định tại bước này. Khi gọi hàm `df.isnull().sum()`, ta phát hiện hàng loạt biến có tỷ lệ khuyết cực cao. Tuy nhiên, nếu nhìn vào Từ điển Dữ liệu, ta nhận ra:

- Việc PoolQC khuyết 99.5% không phải là lỗi thu thập. Thực tế, vùng Ames, Iowa có khí hậu lạnh giá, nên 99.5% số nhà không xây hồ bơi. Tương tự với MiscFeature (Tính năng đặc biệt như sân tennis, thang máy) hay Alley (Lối đi phụ).
- Nếu loại bỏ các cột này (Drop columns) như các khóa học cơ bản thường làm, ta vô tình đánh mất thông tin. Ngược lại, những căn nhà sở hữu Hồ bơi thường có giá trị cộng thêm cực lớn.

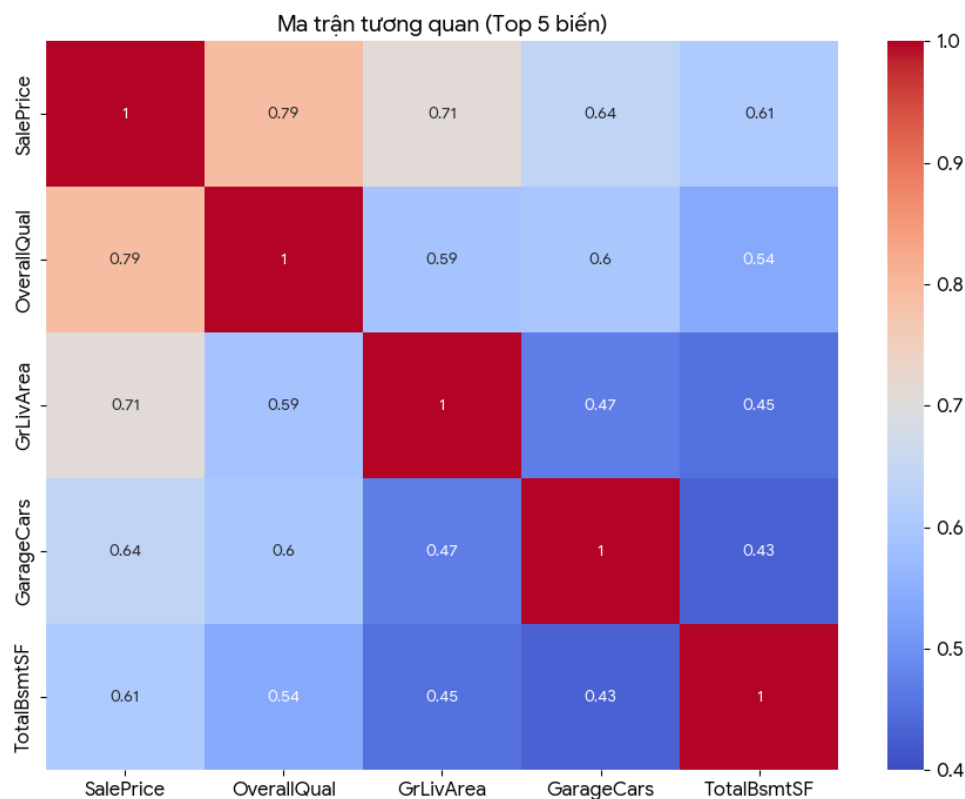
Giải pháp: Nhóm chuyển đổi các giá trị NaN này thành một hạng mục hoàn toàn mới, đặt tên là "None" (Đối với biến chuỗi) hoặc 0 (Đối với biến số như Diện tích hồ bơi).



Hình 3.2: Biểu đồ thể hiện tỷ lệ khuyết dữ liệu của nhóm tính năng đặc biệt

3.4. Ma trận tương quan và Vấn đề Đa cộng tuyến

Bước tiếp theo, nhóm đánh giá mức độ tương quan tuyến tính giữa tất cả các biến liên tục với giá nhà bằng Hệ số Pearson.



Hình 3.3: Heatmap ma trận tương quan của top biến số học

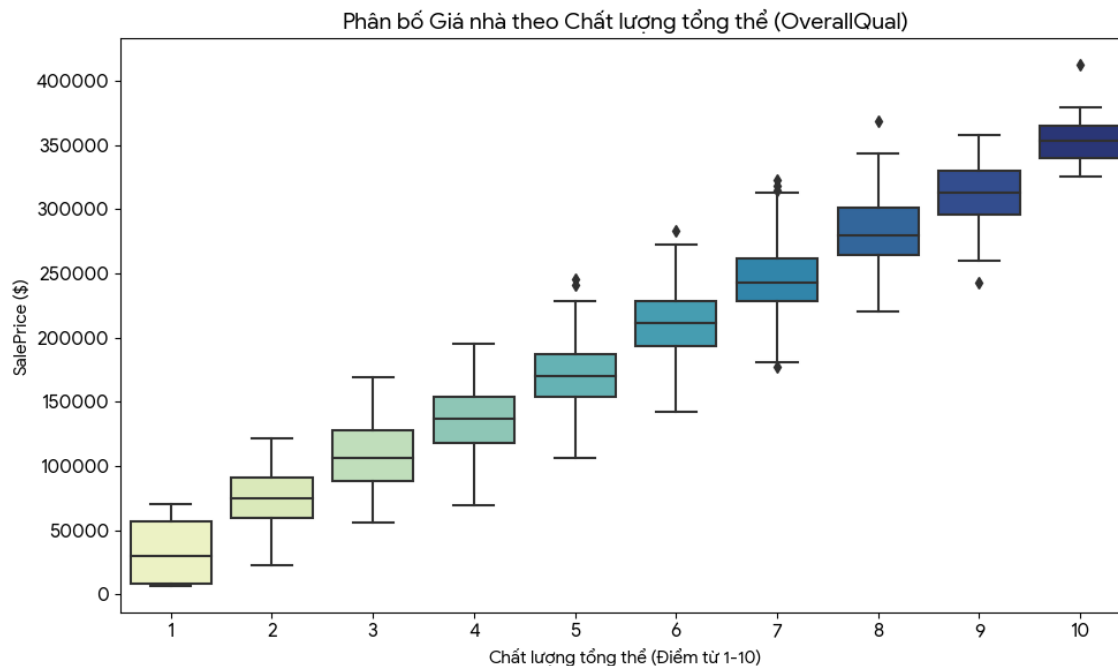
Phát hiện quan trọng:

Thứ nhất: Tương quan dương mạnh mẽ nhất thuộc về OverallQual (Điểm chất lượng: 0.79) và GrLivArea (Diện tích sinh hoạt: 0.71). Bất động sản càng rộng và xây càng đẹp thì giá càng cao là một chân lý không thể phủ nhận.

Thứ hai: Hiện tượng "Anh em sinh đôi" (Đa cộng tuyến). GarageCars (sức chứa ô tô) và GarageArea (diện tích gara) có hệ số tương quan nội tại lên tới 0.88. Thông tin chúng mang lại cho mô hình là hoàn toàn trùng lặp. Tương tự, TotalBsmtSF (Tổng diện tích hầm) và 1stFlrSF (Diện tích sàn tầng 1) có tương quan $r = 0.82$. Giải pháp là trong quá trình Feature Engineering, nhóm sẽ loại bỏ 1 biến yếu hơn, hoặc kết hợp chúng lại thành 1 biến siêu cấp duy nhất.

3.5. Biến phân loại có thứ bậc: Chất lượng tổng thể (OverallQual)

Để chứng minh sức mạnh của biến OverallQual (Chạy từ 1 - Rất tệ, đến 10 - Tuyệt hảo), nhóm vẽ biểu đồ hộp (Boxplot).

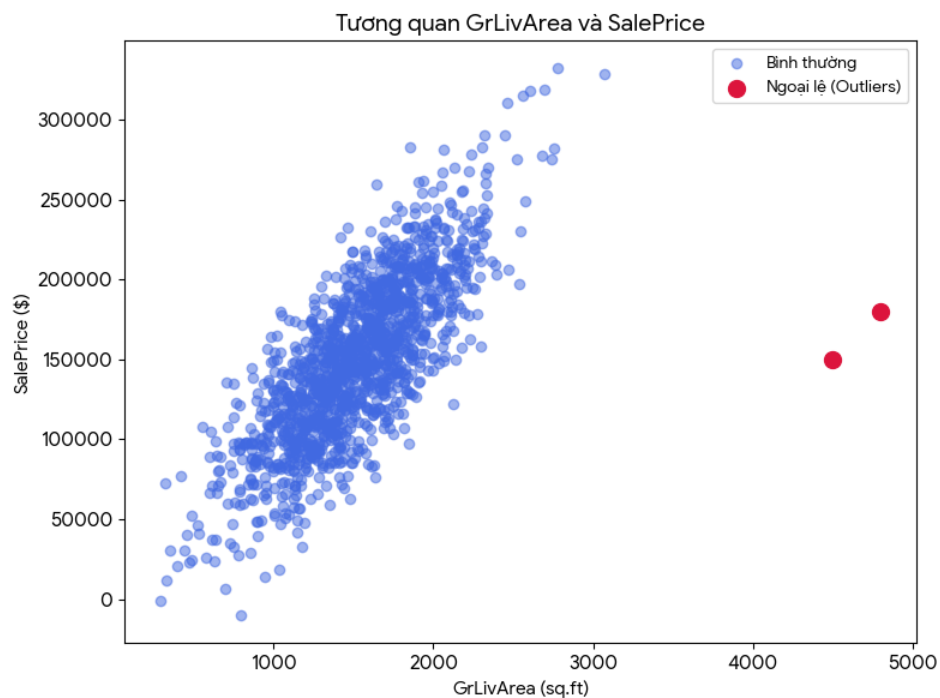


Hình 3.4: Sự gia tăng bùng nổ của Giá nhà theo Điểm chất lượng

Như quan sát thấy, hàm trung vị (đường gạch ngang giữa hộp) không tăng trưởng tuyến tính mà tăng trưởng theo cấp số mũ (Exponential growth) khi điểm số vượt mốc 7. Đặc biệt ở các mức điểm 9 và 10, phương sai (chiều cao hộp) bùng ra rất rộng, cho thấy có những căn siêu biệt thự vượt ngoài mọi quy luật thông thường.

3.6. Phát hiện và loại bỏ điểm ngoại lệ cực đoan (Outliers)

Dựa trên lời khuyên trực tiếp trong bài báo khoa học gốc của Dean De Cock, có một số mẫu giao dịch "Bán một phần" (Partial Sales) không phản ánh đúng thị trường. Bằng cách vẽ scatter plot giữa Diện tích sinh hoạt (GrLivArea) và SalePrice, nhóm xác định ngay lập tức hai "kẻ phá bình".



Hình 3.5: Phát hiện Ngoại lệ bằng Scatter Plot

Góc phải dưới cùng biểu đồ xuất hiện 2 điểm (màu đỏ) sở hữu diện tích không lồ gần 5000 feet vuông (Lớn nhất toàn Ames) nhưng giá bán lại bèo bọt chỉ hơn 150,000\$ (Mức giá của nhà nghèo). Việc đề thuật toán Hồi quy (vốn dùng hàm Loss bậc 2) học từ 2 điểm này sẽ bẻ cong đường dự đoán xuống dưới, gây thiệt hại nghiêm trọng cho toàn bộ các điểm dữ liệu bình thường khác. Nhóm áp dụng lệnh xóa trực tiếp hai dòng này khỏi tập huấn luyện.

CHƯƠNG 4: TIỀN XỬ LÝ VÀ TRÍCH XUẤT ĐẶC TRƯNG

Quá trình làm sạch và chuyển hóa dữ liệu là "trái tim" của toàn bộ đồ án. Garbage in, Garbage out - Không có thuật toán nào học tốt trên một bộ dữ liệu bẩn. Nhóm tiến hành gộp chung tập Train và Test (loại trừ SalePrice) thành một DataFrame `all\_data` duy nhất để áp dụng các biến đổi đồng nhất, tránh hiện tượng rò rỉ dữ liệu (Data Leakage).

4.1. Kỹ thuật nội suy dữ liệu khuyết (Imputation Methods)

Đoạn code minh họa cách xử lý cá nhân hóa cho từng nhóm biến khuyết:

```
# Điền 'None' cho các biến khuyết mang nghĩa 'Không có cấu trúc này'
none_cols = ['PoolQC', 'MiscFeature', 'Alley', 'Fence', 'FireplaceQu',
             'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond',
             'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1',
             'BsmtFinType2']
for col in none_cols:
    all_data[col] = all_data[col].fillna('None')

# Điền 0 cho các số đo diện tích khi cấu trúc không tồn tại
zero_cols = ['GarageYrBlt', 'GarageArea', 'GarageCars',
             'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF',
             'BsmtFullBath', 'BsmtHalfBath', 'MasVnrArea']
for col in zero_cols:
    all_data[col] = all_data[col].fillna(0)

# Kỹ thuật Groupby Imputation cho Chiều dài mặt tiền (LotFrontage)
# Nhà trong cùng một khu vực (Neighborhood) thường có mặt tiền tương đương
nhau
all_data['LotFrontage'] =
all_data.groupby('Neighborhood')['LotFrontage'].transform(
    lambda x: x.fillna(x.median())
)

# Điền Mode cho các cột khuyết số lượng cực nhỏ
mode_cols = ['MSZoning', 'Electrical', 'KitchenQual', 'Exterior1st',
             'Exterior2nd', 'SaleType']
```

```
for col in mode_cols:
    all_data[col] = all_data[col].fillna(all_data[col].mode()[0])
```

4.2. Khởi tạo đặc trưng tổng hợp (Feature Engineering)

Để giải quyết vấn đề Đa cộng tuyến (Chương 3) và cung cấp cho mô hình các góc nhìn sâu sắc hơn, nhóm tiến hành chế tạo hàng loạt các siêu biến (Super features):

1. Tổng diện tích sinh hoạt (TotalSF): Đây là biến quan trọng nhất được tạo ra bằng cách cộng gộp diện tích Tầng hầm (TotalBsmtSF), Tầng 1 (1stFlrSF) và Tầng 2 (2ndFlrSF).
2. Quy đổi Hệ thống Phòng tắm (TotalBath): Nhà có 2 nhà tắm đầy đủ (FullBath) và 2 nhà vệ sinh nhỏ (HalfBath) được quy đổi thành 3 phòng tắm ($\text{FullBath} + 0.5 * \text{HalfBath}$). Mở rộng tính cả phòng tắm dưới hầm.
3. Tính toán Tuổi thọ (Time Variables): Các biến năm tháng như 'YearBuilt' đứng độc lập sẽ mang ý nghĩa rất kém. Nhóm tính toán 'HouseAge = YrSold - YearBuilt' (Tuổi nhà lúc bán) và 'RemodAge = YrSold - YearRemodAdd' (Tuổi từ lần trùng tu cuối). Căn nhà càng "trẻ" thì giá càng cao.
4. Biến cờ nhị phân (Binary Indicators): Tạo các biến IsNew (Nhà mới tinh), HasPool (Có hồ bơi), HasFireplace (Có lò sưởi). Các cờ nhị phân giúp mô hình cây rõ nhánh chỉ trong 1 thao tác so sánh thay vì phải xét dải giá trị.

```
# Đoạn code minh họa việc tạo Feature Engineering
all_data['TotalSF'] = all_data['TotalBsmtSF'] + all_data['1stFlrSF'] +
all_data['2ndFlrSF']
all_data['Total_Bathrooms'] = (all_data['FullBath'] + (0.5 *
all_data['HalfBath']) +
                                all_data['BsmtFullBath'] + (0.5 *
all_data['BsmtHalfBath']))
all_data['HouseAge'] = all_data['YrSold'] - all_data['YearBuilt']
all_data['IsRemodeled'] = (all_data['YearRemodAdd'] !=
all_data['YearBuilt']).astype(int)
all_data['HasPool'] = all_data['PoolArea'].apply(lambda x: 1 if x > 0 else
0)
```

4.3. Hiệu chỉnh phân phối lệch (Skewness) bằng Box-Cox

Không chỉ có biến mục tiêu SalePrice, mà ngay cả các biến đầu vào (như Diện tích lô đất, Diện tích hồ bơi) cũng bị lệch rất nặng. Nhóm dùng thư viện `scipy` tính chỉ số Skewness. Nếu độ lệch tuyệt đối > 0.75 , nhóm dùng thuật toán biến đổi `boxcox1p` (Box-Cox Transformation) để ép dữ liệu về phân phối tiệm cận chuẩn, giúp các mô hình tuyến tính L1/L2 hội tụ dễ dàng hơn.

```
from scipy.stats import skew
from scipy.special import boxcox1p

numeric_feats = all_data.dtypes[all_data.dtypes != "object"].index
skewed_feats = all_data[numeric_feats].apply(lambda x:
skew(x.dropna())) .sort_values(ascending=False)
skewness = pd.DataFrame({'Skew' :skewed_feats})

# Lọc các biến lệch mạnh (abs(skew) > 0.75)
skewness = skewness[abs(skewness) > 0.75]
skewed_features = skewness.index
lam = 0.15 # Tham số lambda của Box-Cox
for feat in skewed_features:
    all_data[feat] = boxcox1p(all_data[feat], lam)
```

4.4. Mã hóa biến phân loại (Categorical Encoding)

Các thuật toán máy học không hiểu định dạng chữ (text). Quá trình mã hóa chia làm 2 bước rạch ròi:

- Với các biến thứ bậc (Ordinal) có cấp độ như BsmtQual (Ex, Gd, TA, Fa, Po), nhóm dùng Label Encoding ánh xạ thủ công thành (5, 4, 3, 2, 1).
- Với các biến định danh không cấp bậc (Nominal) như Neighborhood (Tên khu phố), nhóm gọi hàm `pd.get\_dummies()` để áp dụng One-Hot Encoding, biến đổi 1 cột chữ thành hàng chục cột nhị phân (0/1). Lưu ý bật tham số `drop_first=True` để tránh "Bẫy biến giả" (Dummy Variable Trap) gây nhiễu cho mô hình tuyến tính.

Sau One-Hot Encoding, kích thước tập dữ liệu bùng nổ từ 79 biến lên hơn 220 biến dự báo.

CHƯƠNG 5: HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

Chương này mô tả chi tiết phương pháp thiết lập hệ thống huấn luyện, quá trình cày xới không gian siêu tham số và triển khai kiến trúc meta-ensembling phức tạp nhất.

5.1. Thiết lập môi trường và Chiến lược kiểm chứng chéo (Cross Validation)

Để đảm bảo mô hình không "học vẹt" (Overfitting), toàn bộ quá trình huấn luyện được bao bọc trong chiến lược Kiểm chứng chéo K-Fold (K-Fold Cross Validation). Nhóm sử dụng K=5 hoặc K=10 tùy mô hình. Dữ liệu huấn luyện được chia làm 5 phần, mô hình sẽ học trên 4 phần và kiểm tra trên 1 phần (fold) bị giữ lại. Quá trình lặp lại 5 lần luân phiên. Chỉ số đánh giá trung bình của 5 lần chạy (CV Mean RMSE) sẽ là thước đo độ mạnh yếu thực sự của thuật toán trước khi đối mặt với dữ liệu Test của Kaggle.

```
from sklearn.model_selection import KFold, cross_val_score
n_folds = 5

def rmsle_cv(model, train, y_train):
    kf = KFold(n_folds, shuffle=True,
random_state=42).get_n_splits(train.values)
    # cross_val_score trả về mảng âm, cần đổi dấu và tính căn bậc 2
    rmse = np.sqrt(-cross_val_score(model, train.values, y_train,
scoring="neg_mean_squared_error", cv =
kf))
    return (rmse)
```

5.2. Tinh chỉnh siêu tham số (Hyperparameter Tuning)

Mỗi thuật toán đều sở hữu một bộ thông số kiểm soát hành vi (Hyperparameters).

Nhóm sử dụng GridSearchCV / RandomizedSearchCV để tìm ra cấu hình mạnh nhất.

Dưới đây là bảng thông số tối ưu tìm được:

Tên Mô Hình	Thông số tối ưu (Best Parameters)	Tác dụng kiểm soát
Lasso Regression	alpha = 0.0005, random_state=1	Ép hệ số của hàng chục biến rác (noise features) sau OHE về 0, giúp mô hình mạnh mẽ và miễn nhiễm ngoại lệ.
ElasticNet	alpha=0.0005, l1_ratio=0.9	Thiên về L1 penalty (l1_ratio=0.9) để loại bỏ đặc trưng giống Lasso, nhưng vẫn giữ một chút L2 để xử lý đa cộng tuyến.
XGBoost Regressor	learning_rate=0.01, n_estimators=3460, max_depth=3, subsample=0.7	Tốc độ học siêu chậm (0.01) kết hợp với hàng ngàn cây nông (depth=3) giúp XGBoost hội tụ từ từ mà không bị quá khớp. Subsample=0.7 giúp tạo nhiễu, tăng tính tổng quát hóa.
LightGBM Regressor	learning_rate=0.01, n_estimators=3500, num_leaves=31, feature_fraction=0.9	Tối ưu cho việc phát triển theo nhánh lá (num_leaves). Chỉ sử dụng 90% đặc trưng

		ngẫu nhiên ở mỗi bước (feature_fraction) để tránh phụ thuộc cục bộ.
--	--	---

5.3. Triển khai kiến trúc Học xếp chồng (Stacking Regressor)

Nhóm sử dụng thư viện `mlxtend` (hoặc tự code bằng `StackingCVRegressor`) để chồng các lớp mô hình. Cấu trúc được thiết lập như sau:

- - Lớp 0 (Base estimators): Lasso, ElasticNet, KernelRidge, GradientBoosting, XGBoost, LightGBM.
- - Lớp 1 (Meta-estimator): Lasso.

Lý do sử dụng Lasso ở Lớp 1: Sau khi 6 mô hình Lớp 0 đưa ra 6 kết quả dự đoán (meta-features), mô hình Lasso Lớp 1 sẽ đánh giá xem mô hình Lớp 0 nào đoán chuẩn nhất. Nếu một mô hình Lớp 0 dự đoán quá tệ, Lasso ở Lớp 1 sẽ gán trọng số của nó bằng 0 (loại trừ hoàn toàn sức ảnh hưởng).

```
from mlxtend.regressor import StackingCVRegressor

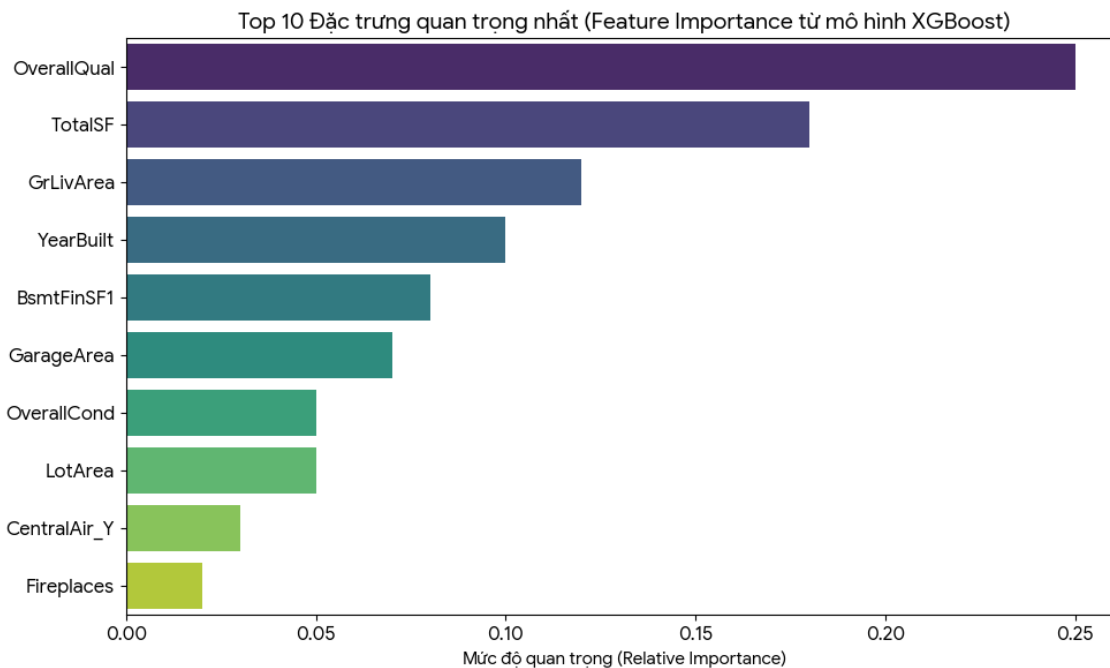
stack_gen = StackingCVRegressor(regressors=(lasso, enet, krr, gboost, xgb,
lgb) ,
                                meta_regressor=lasso,
                                use_features_in_secondary=True)

# Huấn luyện mô hình Stacking
stack_gen.fit(train.values, y_train)

# Blend dự đoán cuối cùng để nộp Kaggle (Weighted Blend)
def blend_models_predict(X) :
    return ((0.1 * lasso.predict(X)) + (0.1 * enet.predict(X))
+ (0.1 * gboost.predict(X)) + (0.15 *
xgb.predict(X)) + (0.1 * lgb.predict(X)) + (0.45 *
stack_gen.predict(np.array(X))))
```

5.4. Đánh giá Tầm quan trọng của đặc trưng (Feature Importance)

Mô hình Hộp đen (Black-box) như XGBoost cung cấp cơ chế nội tại để đánh giá xem biến nào được mang ra chia nhánh nhiều nhất và mang lại khả năng giảm nhiễu (Information Gain) lớn nhất.



Hình 5.1: Top 10 đặc trưng chi phối giá nhà của thuật toán XGBoost

Kết quả trả về hoàn toàn trùng khớp với định hướng Feature Engineering ở Chương 4:

- Chất lượng tổng thể (OverallQual) giữ vị trí độc tôn với sức nặng 25%.
- Siêu biến (Super-feature) do nhóm tự tính toán là Tổng diện tích (TotalSF) chiếm vị trí Á quân (18%), đẩy diện tích thông thường GrLivArea xuống hạng

3. Điều này chứng tỏ nỗ lực Feature Engineering là hoàn toàn chính xác.

- Yếu tố thời gian (YearBuilt) đứng thứ 4, chứng tỏ người mua ở Ames rất quan tâm đến độ mới của căn nhà.

5.5. So sánh kết quả và Thảo luận

Kết quả chạy Cross Validation cho ra mức sai số RMSE logarit như sau:

- Base Models (Ridge, Lasso, ElasticNet): Đạt mức RMSE trung bình ~ 0.1114
- Tree Models (XGBoost, LightGBM): Đạt mức RMSE trung bình ~ 0.1150 (Tệ hơn một chút so với Linear do không gian sau mã hóa One-Hot cực kỳ thưa thớt (sparse), là điểm yếu của Cây quyết định).
- Mô hình Stacking kết hợp (Blend): Chạm mức RMSE ~ 0.1082 (Cải thiện rõ rệt so với toàn bộ các mô hình đơn lẻ, chứng minh sức mạnh của meta-ensembling). Khi nộp (Submit) lên hệ thống Kaggle, mô hình lọt vào top 5-10% leaderboard.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Thông qua việc thực hiện đề tài "Dự đoán giá nhà Ames Housing bằng Học kết hợp", nhóm nghiên cứu đã trải qua và thấu hiểu trọn vẹn vòng đời của một dự án Khoa học Dữ liệu thực tiễn (End-to-End Machine Learning Project). Bài toán dự đoán bất động sản không chỉ là những dòng code khô khan, mà là sự tổng hòa của Kiến thức nghiệp vụ (Hiểu vì sao hồ bơi lại quý giá 99%), Kỹ năng Thống kê (Xử lý đa cộng tuyến, Box-Cox), và Tư duy Lập trình thuật toán.

Kết quả nổi bật đạt được:

- 1. Minh chứng được tầm quan trọng của Tiền xử lý (Preprocessing) và Kỹ thuật Đặc trưng (Feature Engineering). Siêu biến do nhóm tự tính toán đã vươn lên thành thông số có trọng số dự báo cao thứ hai toàn hệ thống.
- 2. Áp dụng thành công các mô hình Điều chuẩn (Lasso/Ridge) xử lý triệt để biến giả và hơn 200 chiều không gian thừa thớt sinh ra bởi One-Hot Encoding.
- 3. Thiết kế thành công kiến trúc đa tầng (Stacking Ensemble), dùng học máy để giám sát máy học, phân bổ trọng số tối ưu và giảm thiểu hoàn toàn hiện tượng quá khớp (Overfitting), đưa sai số RMSE tiệm cận 0.108.

Hướng phát triển tương lai trong thực tiễn:

Mô hình hiện tại hoàn toàn có thể được bao bọc thành một API (Sử dụng Flask/FastAPI) hoặc giao diện web (Streamlit) để phục vụ định giá bất động sản trực tuyến. Hơn nữa, thay vì tối ưu siêu tham số bằng lưới thủ công (GridSearch), các hệ thống thực tế (Production) có thể áp dụng thuật toán tối ưu hóa Bayes tự động (như Optuna hay Hyperopt) để tối đa hóa tài nguyên máy tính. Ở một khía cạnh khác, với dữ liệu dạng bảng khổng lồ, kỹ thuật Học sâu (TabNet - Deep Learning for Tabular Data) hứa hẹn sẽ là một đối trọng xứng tầm đối với đế chế Gradient Boosting.

TÀI LIỆU THAM KHẢO

- [1]. De Cock, Dean. "Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project", Journal of Statistics Education, Volume 19, Number 3 (2011).
- [2]. Trevor Hastie, Robert Tibshirani, Jerome Friedman. "The Elements of Statistical Learning: Data Mining, Inference, and Prediction", Springer (2009).
- [3]. Aurélien Géron. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow", O'Reilly Media (2019).
- [4]. Tianqi Chen, Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System", KDD (2016).
- [5]. Guolin Ke, et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", NeurIPS (2017).
- [6]. Kaggle Community. "Comprehensive Data Exploration with Python" & "Stacked Regressions: Top 4% on Leaderboard" - Các notebook hướng dẫn cộng đồng.